

Lecture 4 — NoSQL Databases and Partitioning: KV, Cache-Aside, Sharding, Documents

Running example data (same employees as the file-formats diagram)

id=1 Alice (Developer, active) · id=2 Bob (Designer, inactive) · id=3 Charlie (Manager, active) · id=4 Dana (Developer, inactive) · id=5 Eve (Designer, active)

1. Key-Value Model — Redis

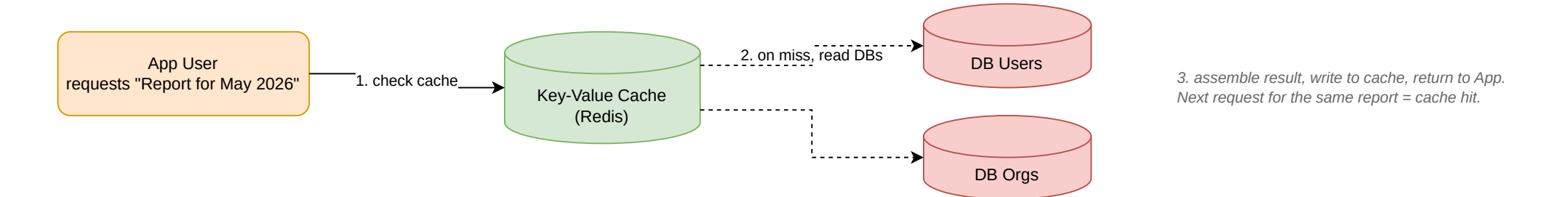
One key → one opaque value (a JSON document). Lookups by key are O(1). The store does not index into the value.

Key	Value (JSON document)
"1"	{ "id": 1, "name": "Alice", "role": "Developer", "active": true }
"2"	{ "id": 2, "name": "Bob", "role": "Designer", "active": false }
"3"	{ "id": 3, "name": "Charlie", "role": "Manager", "active": true }

Example use: a session store (key = session-id, value = user blob), or a product cache (key = product-id, value = page-render JSON).

2. Cache-Aside Pattern — where Key-Value sits in production

The KV store is rarely the system of record. It sits IN FRONT of one. App reads the cache first; on miss, reads the DB(s), assembles the answer, writes it back to the cache.

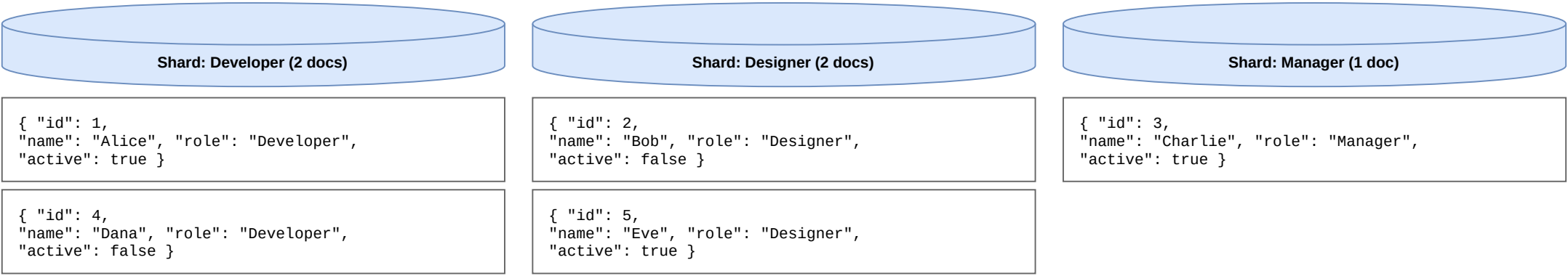


3. Data Partitioning — DynamoDB / MongoDB sharding

The partition KEY decides which physical shard each record lands on. Choice is QUERY-DRIVEN — there is no universally right partition key.

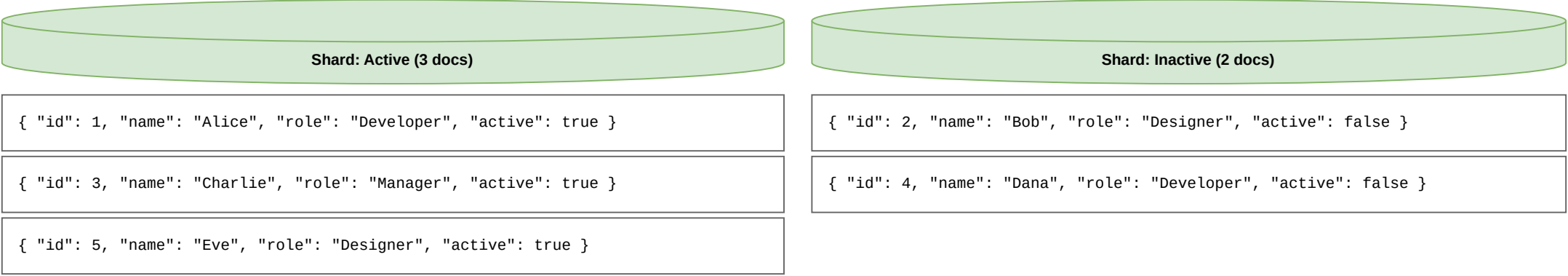
Driving query: "How many ACTIVE DEVELOPERS are there?" → answer is 1 (Alice). The three strategies below differ in how cheaply they answer this. Each shard shows the ACTUAL MongoDB documents that would live there.

3a. Partition by Role (single attribute)



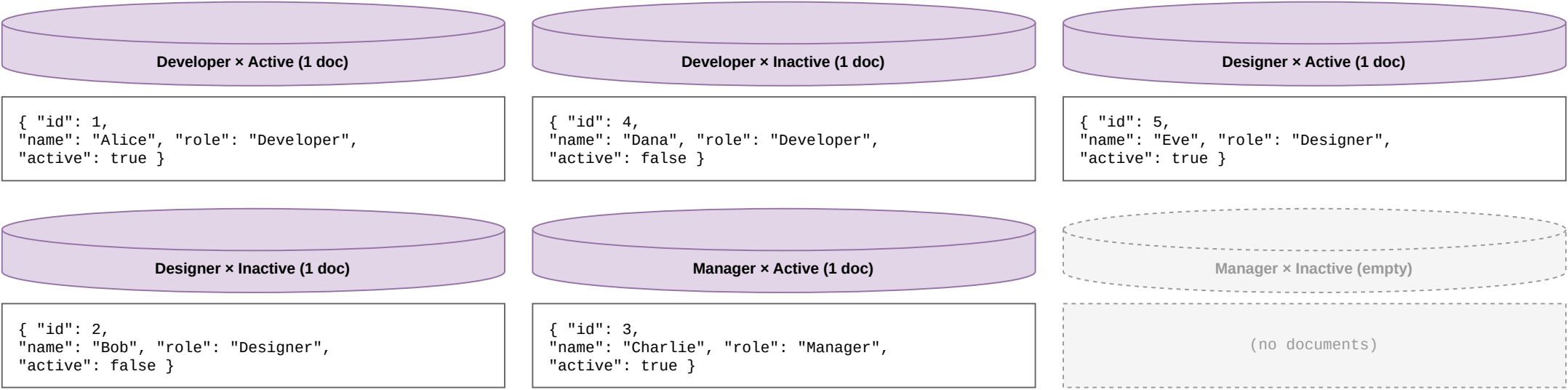
Driving query: read the Developer shard (2 docs), filter active = true. Touches 1 shard but still scans 2 docs to find the 1 answer.

3b. Partition by Active Status (different single attribute)



Driving query: read the Active shard (3 docs), filter role = Developer. Touches 1 shard but scans 3 docs. Worse than 3a for THIS query.

3c. Partition by Role × Active (composite key)



Driving query: read EXACTLY ONE shard (Developer × Active). 1 doc, no filter needed. Best for THIS query.
Trade-off: a query that filters on Active alone (e.g. "all active users") now touches 3 shards. Composite keys are great for the queries they index, worse for queries they don't.

4. Document Stores — MongoDB with embedded sub-documents

Unlike Redis, MongoDB INDEXES INTO the value. A collection holds many documents; each document is a JSON-shaped record; nested sub-documents replace what would be a JOIN in SQL.

Collection: employees (richer documents with embedded address sub-document)

```
{
  "id": 1,
  "name": "Alice",
  "role": "Developer",
  "active": true,
  "joined_date": "2026-05-12",
  "address": {
    "street": "123 Main St.",
    "city": "Newark",
    "zip": "07101"
  }
}
```

```
{
  "id": 2,
  "name": "Bob",
  "role": "Designer",
  "active": false,
  "joined_date": "2024-11-03",
  "address": {
    "street": "45 Oak Ave.",
    "city": "Jersey City",
    "zip": "07302"
  }
}
```

```
{
  "id": 3,
  "name": "Charlie",
  "role": "Manager",
  "active": true,
  "joined_date": "2026-05-20",
  "address": {
    "street": "7 Pine Rd.",
    "city": "Hoboken",
    "zip": "07030"
  }
}
```

Each document has its own EMBEDDED address sub-document — no JOIN to a separate addresses table.

Sample query and compound index

```
QUERY: db.employees.find({ joined_date: { $gte: "2026-05-01" }, active: true })
// "all active users who joined in or after May 2026" → Alice, Charlie

INDEX: db.employees.createIndex({ joined_date: 1, active: 1 })
// compound index on (joined_date, active) – makes the query O(log n) instead of full collection scan
```